
Neuroinformatics project

Name: Rezvan Afshoon

Marticulation Number: 1017747

Introduction

- **Goal**

The objective of this project is to develop a machine learning model that can classify whether a stimulus presented to the brain is familiar or unfamiliar, based on the firing patterns of neurons recorded during that stimulus.

- **Data**

We use a dataset consisting of 800 samples, where each sample contains the spiking activity of 1000 neurons recorded over 501 time steps. This forms a 3D data array with shape (800, 1000, 501). Each trial represents one stimulus event and its corresponding neural response.

- **Task**

The task is to process this raw spike train data into a format suitable for machine learning, extract meaningful features, and train a binary classifier to predict if the stimulus was familiar (label 1) or unfamiliar (label 0). This involves neural coding techniques, model selection, and performance evaluation.

Data

- 800 samples → 800 unique experiments (stimuli presentations)
- 1000 neurons → For each sample, the firing activity of 1000 neurons is recorded
- 501 timesteps → Each experiment records neuron spikes over 501 time bins

Target Labels (y)

- Binary classification target:
 - $y[i] = 1$ → The stimulus was familiar to the neurons
 - $y[i] = 0$ → The stimulus was unfamiliar (novel)

```
[6] print("x shape:", x.shape)
    print("y shape:", y.shape)
```

```
↔ x shape: (800, 1000, 501)
  y shape: (800,)
```

Data Preprocessing

✓ 1. Rate Coding (Average spike rate per neuron)

We want to see how active each neuron is. The higher the firing rate, the stronger the neuron's response to the stimulus. We count how many spikes a neuron fires, and then divide it by the total time. Then, we have **one feature vector** per trial.

```
[ ] firing_rate_features = X.mean(axis=2)
    print(firing_rate_features.shape)
```

```
↩ (800, 1000)
```

✓ 2. Total spike count

Here we measure how active the whole brain network is together. Instead of looking at each neuron individually, here we sum up all the spikes fired by all neurons across all time steps in one trial. This gives us **one number** per trial.

```
[ ] features_total_spikes = X.sum(axis=(1, 2)).reshape(-1, 1)
```

We transform raw spike train data into structured feature vectors suitable for machine learning models.

Methods Used

1. Rate Coding

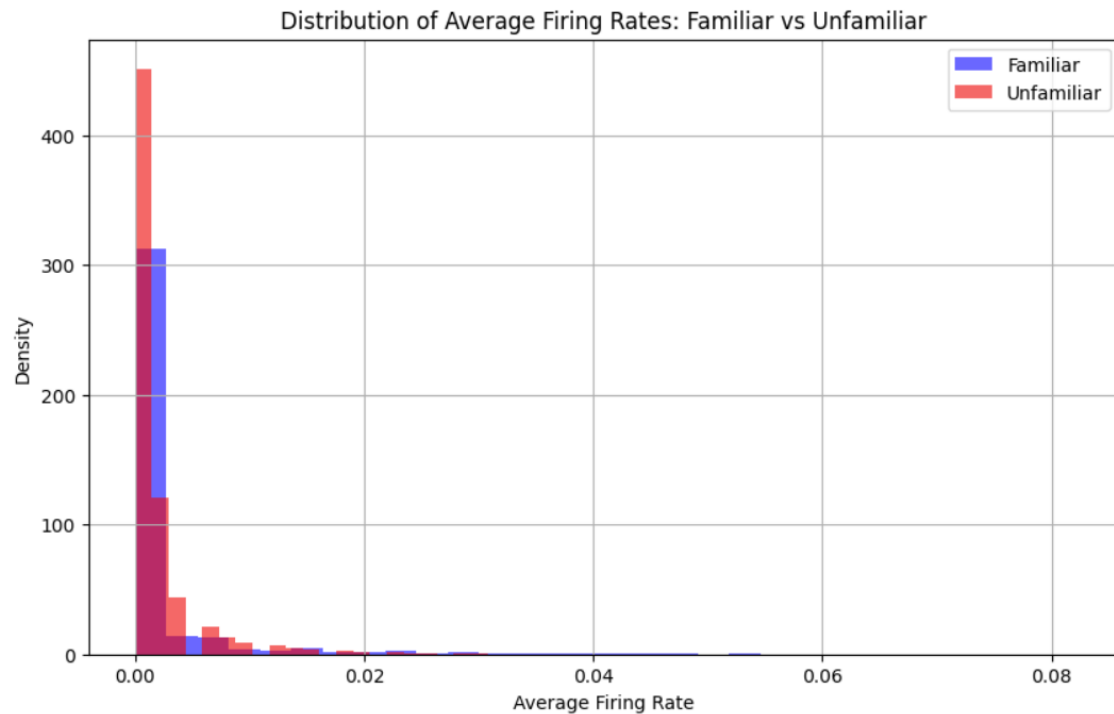
Calculated the average firing rate per neuron across all time steps.

Captures how active each neuron was in response to the stimulus.

2. Total Spike Count

- Computed the total number of spikes per trial across all neurons and time
- Summarizes overall brain activity intensity.

Exploratory Data Analysis



- In this step, we plot the distributions of the extracted feature (average firing rates) for familiar and unfamiliar stimuli.
- Out of the model comparison metrics we studied during the course, we chose to use **Kullback-Leibler (KL) Divergence** here because it is specifically designed for comparing two probability distributions. KL Divergence measures how much one distribution diverges from another, making it ideal for analyzing whether the two classes (familiar vs unfamiliar) are separable based on the features.
- The calculated KL Divergence between the familiar and unfamiliar distributions is **4.53**. This high value indicates a strong divergence between the two distributions, meaning that the feature we extracted (average firing rate) separates the two classes very well

Model 1: Logistic Regression

We chose to use Logistic Regression as the classification model because it is a simple, interpretable, and effective method for binary classification tasks.

Since our goal is to distinguish between two classes (familiar vs unfamiliar stimuli) based on average firing rates, Logistic Regression provides a good baseline with easily interpretable results.

First, we prepare our data to train the model:

```
▶ x_features = firing_rate_features  
y_labels = y
```

Then we split the data into train and test set.

```
[ ] x_train, x_test, y_train, y_test = train_test_split(  
    x_features, y_labels, test_size=0.2, random_state=42, stratify=y_labels  
)
```

We train logistic regression and make predictions

```
[ ] logreg = LogisticRegression(max_iter=1000, random_state=42)  
logreg.fit(x_train, y_train)  
  
y_pred = logreg.predict(x_test)
```

Model Evaluation

- Since the classes in our dataset are approximately balanced, we used **Accuracy** as the primary metric for evaluating the model performance.
- Accuracy measures the proportion of correctly classified samples over the total number of samples. In balanced datasets, Accuracy is a reliable and intuitive metric to assess the overall effectiveness of the classifier. The calculated accuracy score provides a straightforward evaluation of how well the model distinguishes between familiar and unfamiliar stimuli.

```
[ ] accuracy = accuracy_score(y_test, y_pred)
    print(f"Accuracy of the model: {accuracy:.4f}")
```

⇒ Accuracy of the model: 0.7625

- The model achieved an accuracy of **76.25%** on the test set.
- This indicates that the classifier can reliably distinguish between familiar and unfamiliar stimuli based on the extracted average firing rate features.

Model 2: Random Forest

- To compare different models, we use another classification model called Random Forest.
- Unlike logistic regression, it captures nonlinear relationships and automatically identifies the most important features. It achieved better accuracy and showed more robustness to irrelevant or noisy neuron activity.

```
▶ # 1. Split full features as before
X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(
    firing_rate_features, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Train Random Forest Classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_rf, y_train_rf)

# 3. Predict and evaluate
y_pred_rf = rf.predict(X_test_rf)
accuracy_rf = accuracy_score(y_test_rf, y_pred_rf)

print(f"Accuracy using Random Forest: {accuracy_rf:.4f}")
```

⇒ Accuracy using Random Forest: 0.8750

- This model achieved a significantly higher accuracy of **0.870**, outperforming logistic regression.

Model With Fewer Amount of Neurons:

- We trained a logistic regression model using only the first 100 neurons instead of all available neurons. This model achieved an accuracy of **0.600**, indicating a noticeable drop in performance compared to using all neurons (0.7625).
This suggests that limiting the number of neurons removed important information needed for accurate classification.

```
[ ]
# 1. Select fewer neurons
features_rate_small = firing_rate_features[:, :100] # we use only the first 100 neurons

# 2. Split into train/test sets
X_train_small, X_test_small, y_train_small, y_test_small = train_test_split(
    features_rate_small, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Train Logistic Regression
logreg_small = LogisticRegression(max_iter=1000, random_state=42)
logreg_small.fit(X_train_small, y_train_small)

# 4. Predict and evaluate
y_pred_small = logreg_small.predict(X_test_small)
accuracy_small = accuracy_score(y_test_small, y_pred_small)

print(f"Accuracy using fewer neurons (100 features): {accuracy_small:.4f}")
```

➡ Accuracy using fewer neurons (100 features): 0.6000

Comparing Different Models:

Model Comparison Table

Model	Features Used	Classifier	Accuracy
Logistic Regression	All neurons	Logistic Regression	0.7625
Logistic Regression	100 neurons	Logistic Regression	0.6000
Random Forest	All neurons	Random Forest	0.8750

- Using all neurons with Logistic Regression achieved **76.25%** accuracy.
- Reducing to the first 100 neurons dropped accuracy to **60%**, showing that neuron diversity contributes significantly to performance.
- The Random Forest classifier was also implemented to compare model robustness and performance and the accuracy raised to **87%**.